

Oracle

An oracle is a gate-level circuit that encodes a function $f(x) \rightarrow \{0,1\}$. An oracle is like a lock. You insert a key (input qubit), and the lock either clicks or doesn't (output qubit flips or stays). You don't see the mechanism inside — but the mechanism is real. Oracle is nothing but a combination of gates - CNOT, X gates etc. .

In this section, we will see the role of oracles in the Deutsch algorithm.

The problem it solves

Given $f: \{0,1\} \rightarrow \{0,1\}$, determine whether f is *constant* ($f(0) = f(1)$) or *balanced* ($f(0) \neq f(1)$). Classically this needs 2 queries. The Deutsch algorithm does it in 1 query, by exploiting superposition and phase kickback. If the output is 0 then function is constant and if it is 1 then function is balanced.

There are four possibilities of functions $f(x)$ and of oracle also.

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	1	1
1	0	1	0	1

Case	$f(0)$	$f(1)$	Gates
Constant-0	0	0	none
Constant-1	1	1	X on q_1
Balanced (copy)	0	1	CNOT($q_0 \rightarrow q_1$)
Balanced (flip)	1	0	CNOT($q_0 \rightarrow q_1$), X on q_1

Note that in this section, our major focus is implementation of oracles. If you want to learn the logic of the Deutsch Jozsa algorithm then visit its section.

Let's implement it with Qiskit.

First we map the problem:

```

1  from qiskit import QuantumCircuit
2  from qiskit_aer import AerSimulator
3
4  def deutsch_oracle(case: str) -> QuantumCircuit:
5      qc = QuantumCircuit(2)
6      if case == "constant_0":
7          pass
8      elif case == "constant_1":
9          qc.x(1)
10     elif case == "balanced_copy":
11         qc.cx(0, 1)
12     elif case == "balanced_flip":
13         qc.cx(0, 1)
14         qc.x(1)
15     return qc
16
17 def run_deutsch(case: str):
18     qc = QuantumCircuit(2, 1)
19     qc.x(1)          # ancilla = |1>
20     qc.h(0)
21     qc.h(1)
22     qc.barrier()
23     qc.compose(deutsch_oracle(case), inplace=True)
24     qc.barrier()
25     qc.h(0)
26     qc.measure(0, 0)
27

```

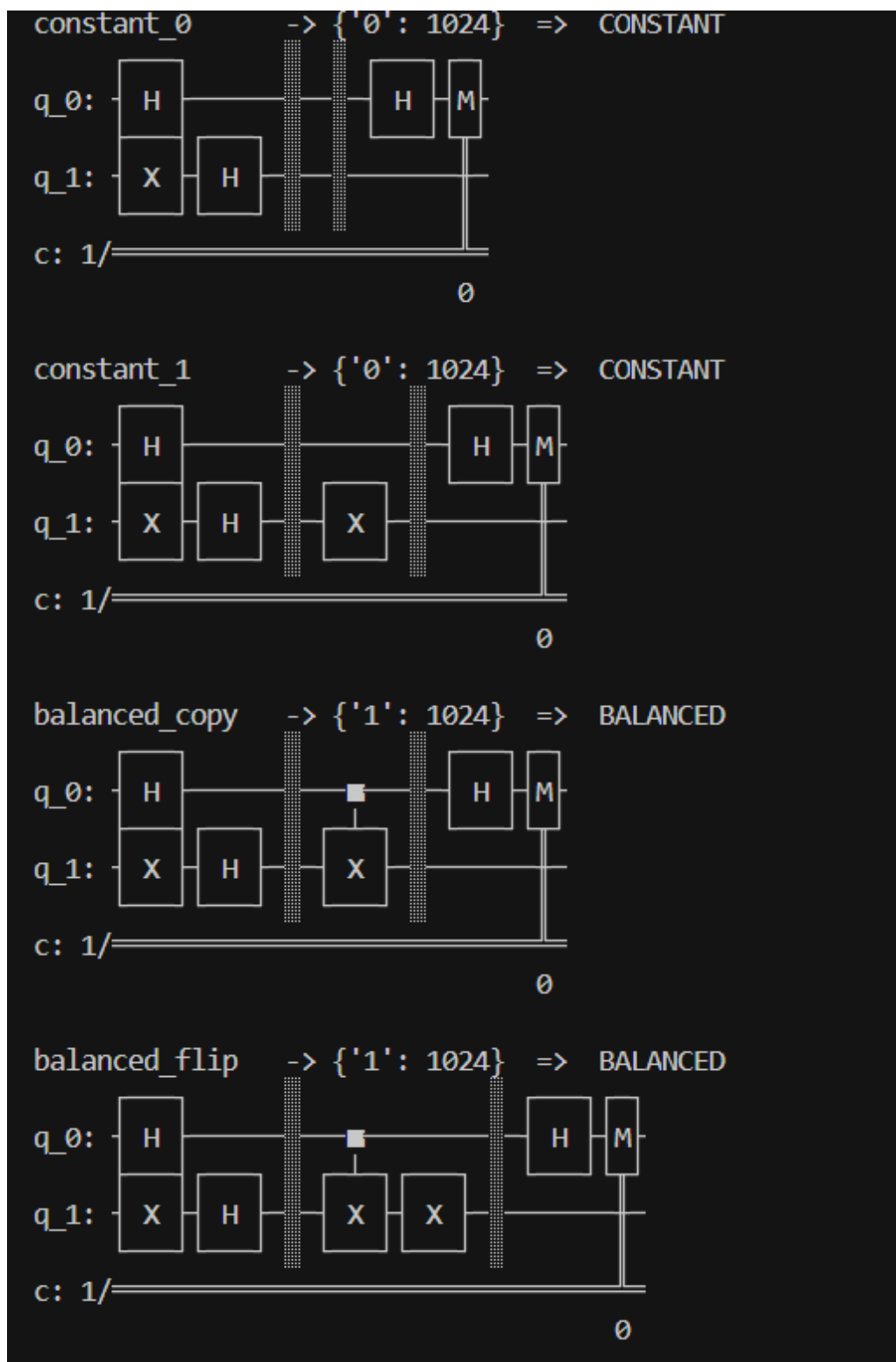
Now , run and visualize the circuit .

```

28     sim = AerSimulator()
29     counts = sim.run(qc, shots=1024).result().get_counts()
30     result = max(counts, key=counts.get)
31     verdict = "CONSTANT" if result == "0" else "BALANCED"
32
33     print(f"{case:15s} -> {counts} => {verdict}")
34     print(qc.draw('text'))
35     print()
36
37 for case in ["constant_0", "constant_1", "balanced_copy", "balanced_flip"]:
38     run_deutsch(case)

```

OUTPUT :



I hope that it has made a clear understanding of oracles . Oracles are also called “ black box “ not because its internals are hidden, but because the algorithm using it does not need to know what's inside — it only reads the effect on an output qubit. There are many algorithms using oracles such as the Grover’ algorithm , Bernstein-Vazirani algorithm etc .

